



华中农业大学
HUAZHONG AGRICULTURAL UNIVERSITY

数据库原理实验 大作业报告

课程名称 : 数据库原理

指导教师 : 王海燕

基于 Java 的小型 DBMS 的开发

题 目 : —Panda 熊猫数据库

班级姓名 : 李莎

学 号 : 2021306220119

专业班级 : 计算机类计算机 2208 班

学 院 : 信息学院

目录

一、 系统简述	1
1.1. 功能概述:	1
1.1.1. 数据库登录界面	1
1.1.2. 小型数据库可视化	1
1.1.3. 数据库操作	1
1.2. 实现思路:	1
1.2.1. 数据库代码逻辑结构	1
1.2.2. 数据库存储结构	2
二、 总体结构与功能设计	2
2.1. 总体结构:	2
2.1.1. 数据库代码逻辑结构:	2
2.1.2. 数据库存储结构:	3
2.2. 功能设计:	3
2.2.1. 可视化部分	3
2.2.2. 数据库 sql 语句部分	3
三、 系统实现与效果展示	5
3.1. 登录界面	5
3.2. 操作界面	5
四、 核心代码	13
4.1. 登录数据库界面	13
4.2. 数据库界面	13
4.3. 数据库 sql 语句操作语句	14
4.3.1. SQL 语句解析	14
4.3.2. 创建数据库	16
4.3.3. 连接数据库	17
4.3.4. 创建表格	18
4.3.5. 插入数据	19
4.3.6. 删除数据	21
4.3.7. 查找数据	22
4.3.8. 更新数据	24
4.4. 工具类	25
4.4.1. Table 类	25
4.4.2. Lines 类	26
4.5. 解决难点	27

一、系统简述

在数据库原理课程学习中，掌握了关系数据库结构和 SQL 语句，激发了我用 Java 搭建一个小型 DBMS 的兴趣。经过学习和编程，我完成了基本 SQL 指令的解析、执行以及数据的离线存储与处理，创建了 Panda 熊猫数据库。以下是该 DBMS 的功能概述和实现方法：

1.1. 功能概述：

1.1.1. 数据库登录界面

模仿 SQL Server 设置用户名和密码，连接服务器。点击账号信息可获取当前的账号信息。输入正确账号信息后，点击连接服务器可以建立与服务器的连接。

1.1.2. 小型数据库可视化

允许用户在界面上输入 SQL 语句。

点击执行命令后，SQL 命令将被传递到操作函数执行，并将执行结果返回到该页面。

1.1.3. 数据库操作

允许用户输入基本的 SQL 语句，如创建数据库、删除数据库、选择数据库、关闭数据库、创建表、删除表、查询数据、插入数据、更新数据、删除数据。

输入的 SQL 语句会被传入操作函数中，经过正则表达式匹配后，执行相应的数据库操作。

1.2. 实现思路：

1.2.1. 数据库代码逻辑结构

用户通过 DBMS 输入 SQL 语句，该语句传递到操作函数。操作函数根据输入的 SQL 语句使用正则表达式进行匹配，然后执行相应的数据库操作。操作函数将数据库操作结果返回到 DBMS。

1.2.2. 数据库存储结构

将数据全部存储到文件中，降低内存占用，支持离线存储与导入，提高可移植性。

二、 总体结构与功能设计

2.1. 总体结构：

2.1.1. 数据库代码逻辑结构：

工作原理是向数据库系统之中输入 SQL 语句，SQL 语句传入操作函数之中，通过操作函数到数据库之中执行对应的操作，数据库之中的操作结果返回到 DBMS 数据库系统之中。

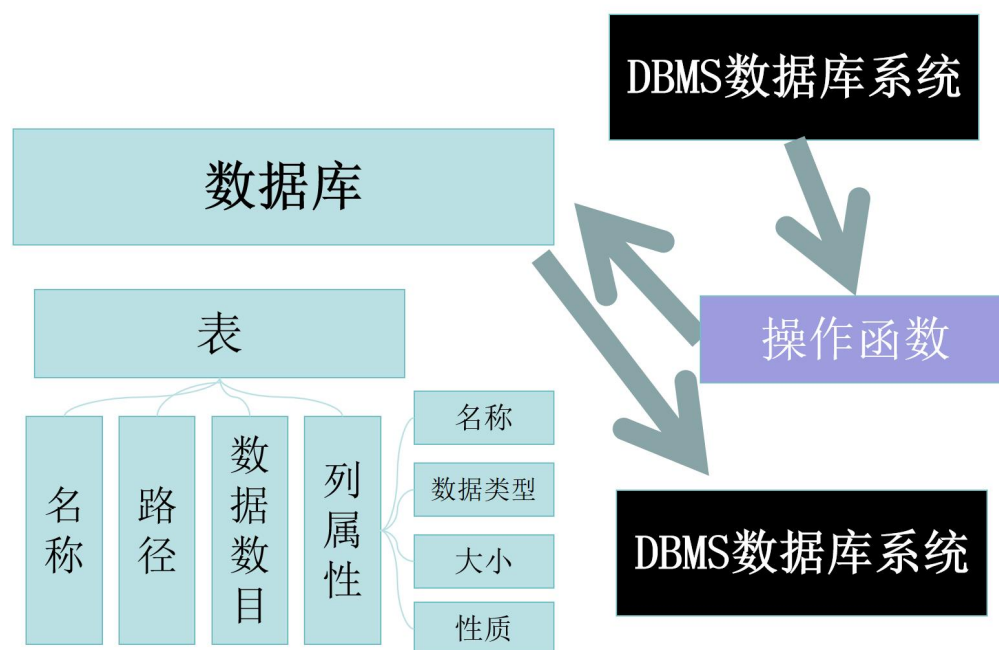


图 1 数据库代码的逻辑结构

2.1.2. 数据库存储结构:

将数据全部存储到文件中，降低内存占用，支持离线存储与导入，提高可移植性。

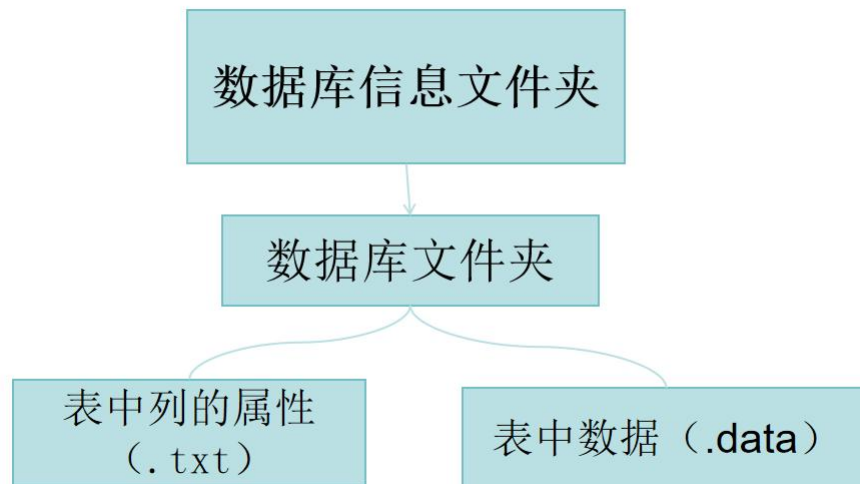


图 2 数据库的储存结构

借鉴 SQL Server 系统的架构，采用了一个表结构+数据存储的方式，实现了表的代码逻辑和物理存储结构。

2.2. 功能设计:

2.2.1. 可视化部分

在登录界面模仿 SQL Server 设置用户名和密码，点击账号信息即可获取当前的账号信息，输入正确账号信息点击连接服务器可以连接到服务器。

在服务器界面输入 SQL 语句点击执行命令，命令执行结果返回到文本显示框。

点击帮助可以使用查看基本语法和清空文本显示框的功能。

2.2.2. 数据库 sql 语句部分

在数据库页面输入 sql 基本语句，将 sql 语句传入操作函数中通

过正则表达式匹配输入的 SQL 命令，然后根据命令执行相应的操作。

可执行的操作的操作包括：创建数据库、删除数据库、选择数据库、关闭数据库、创建表、删除表、查询数据、插入数据、更新数据、删除数据。

创建数据库： `create database 名称;`

删除数据库： `drop database 名称;`

连接数据库： `use 名称;`

关闭数据库： `close 名称;`

创建表： `create table 名称 (列名 数据类型 (大小) 性质, ...);`

删除表： `drop table 名称;`

插入数据： `insert into 表名 (类型 1, 类型 2...) values (v1, v2...);`

修改数据： `update 表名 set 列名=新值 where 条件;`

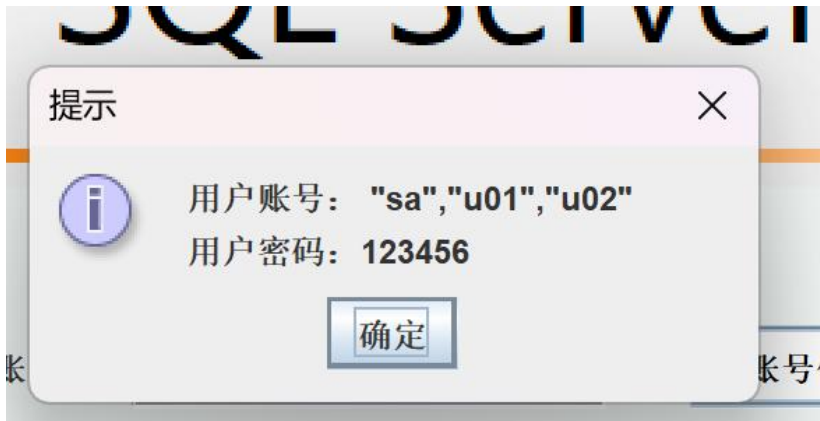
查询数据： `select 列明 from 表名 where 条件;`

删除数据： `delete from 表名 where 条件;`

三、系统实现与效果展示

3.1. 登录界面

查看用户密码



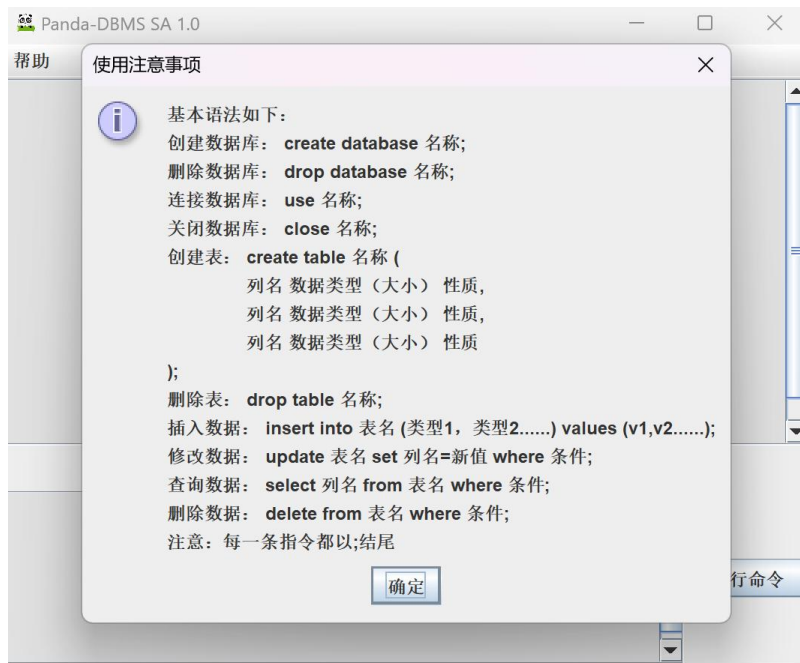
连接到服务器



3.2. 操作界面

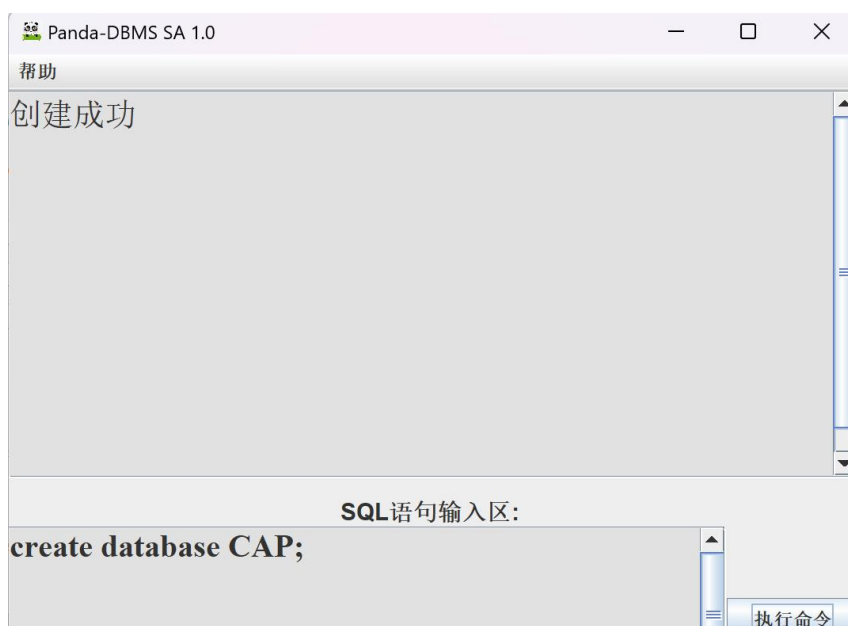
3.2.1. 帮助

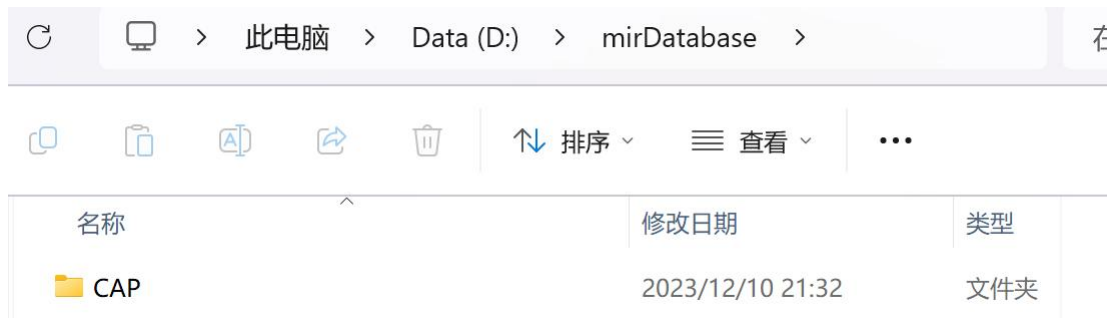
点击帮助的基本语法可以显示该数据库的基本语法或者清空显示区的内容



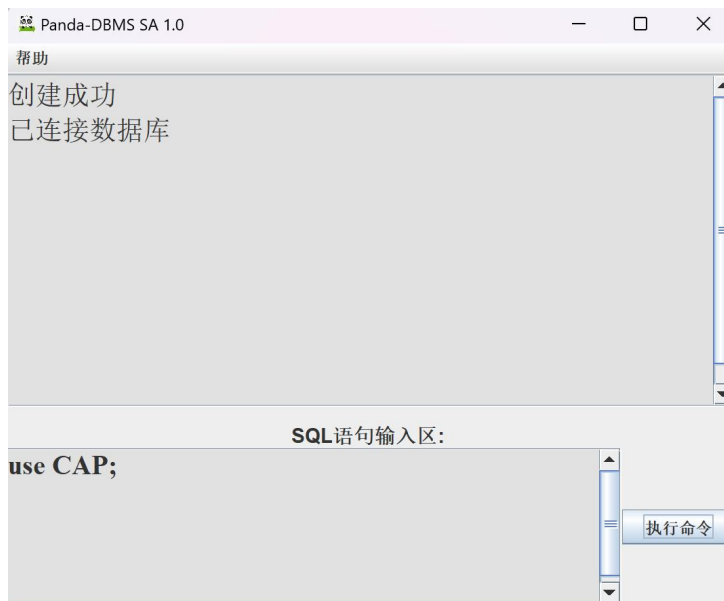
3.2.2. 创建数据库:

在电脑的 D 盘 mirDatabase 文件夹之中创建一个数据库文件夹，储存该数据库的数据



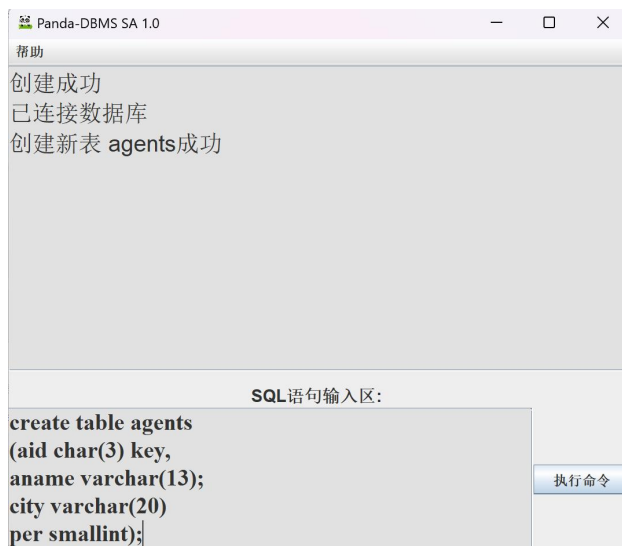


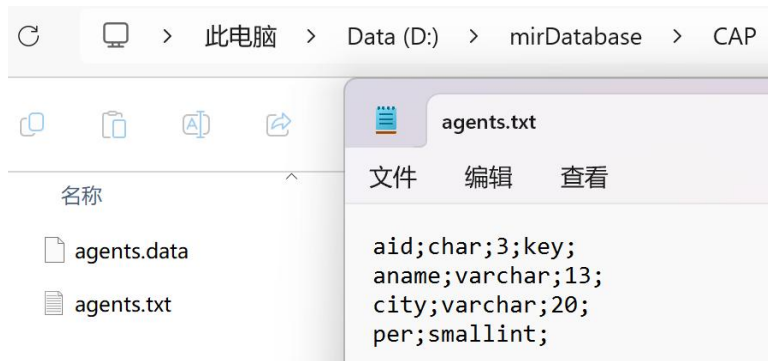
3.2.3. 连接数据库：



3.2.4. 创建表格：

在电脑的 D 盘 mirDatabase 的数据库文件夹之中创建两个文档，txt 文件储存该表的数据属性信息，data 之中储存该表的内容信息

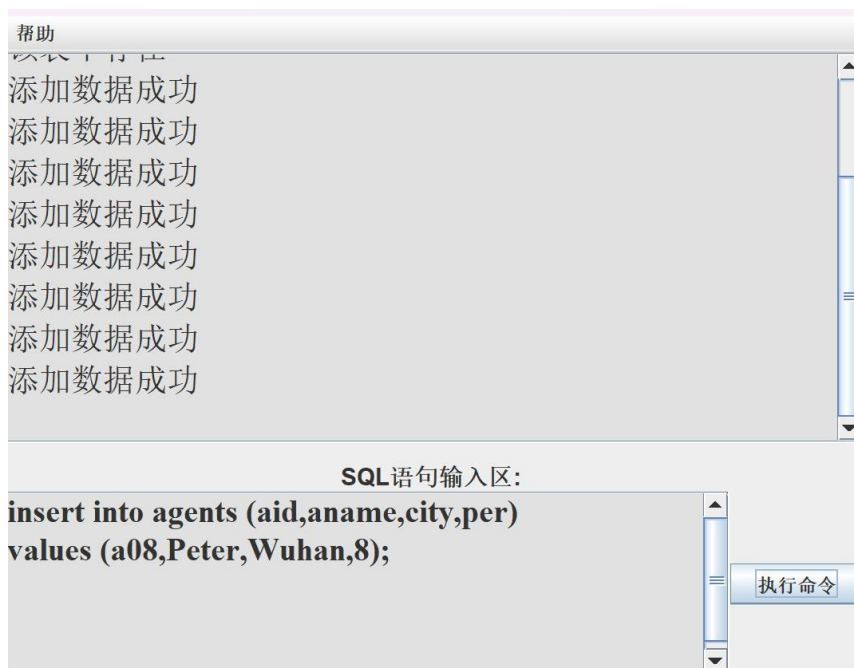




3. 2. 5. 添加数据:

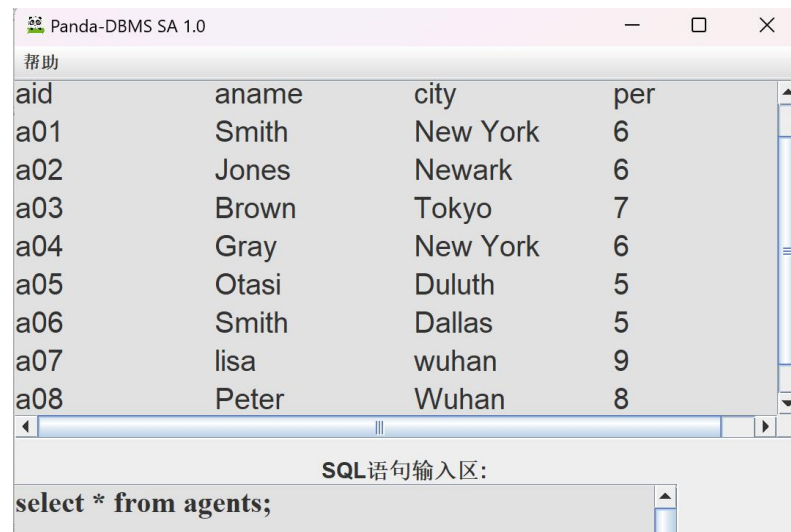
将添加的数据写入在电脑的D 盘 mirDatabase/CAP/agents. data 之中
储存该表的内容信息

agents.data			
文件	编辑	查看	
a01	Smith	New York	6
a02	Jones	Newark	6
a03	Brown	Tokyo	7
a04	Gray	New York	6
a05	Otasi	Duluth	5
a06	Smith	Dallas	5
a07	lisa	wuhan	9
a08	Peter	Wuhan	8

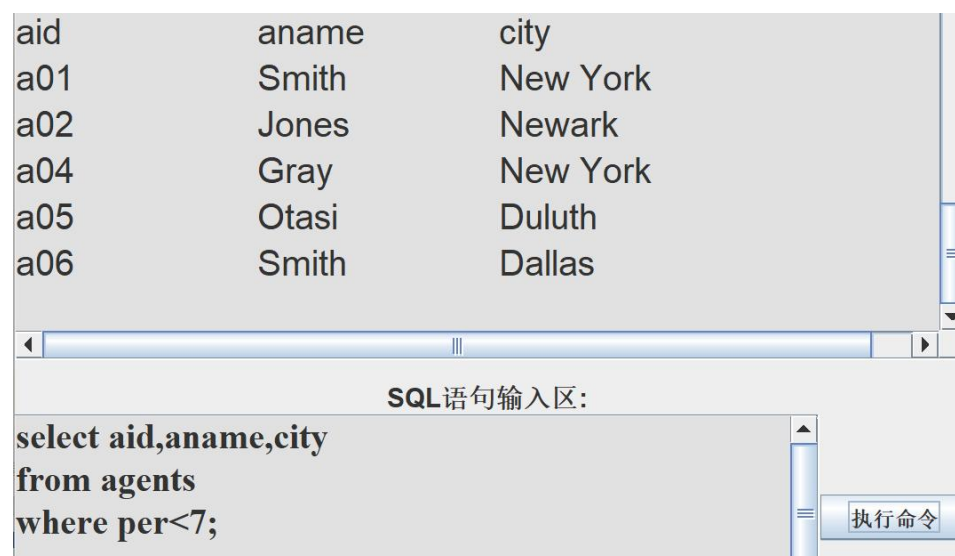


3.2.6. 查询数据：

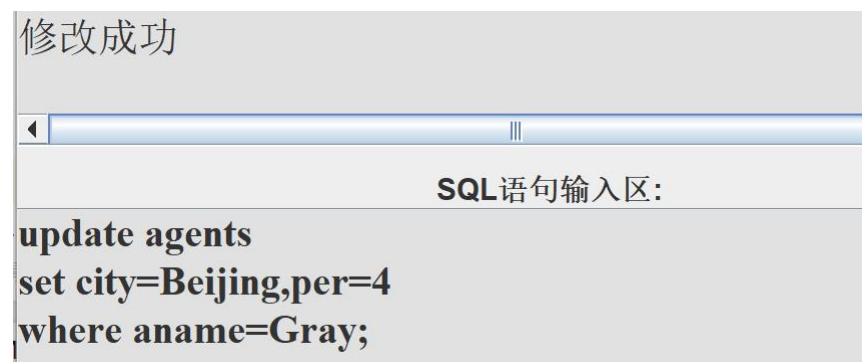
查询所有信息，将查询之后的信息在界面之中显示出来出来



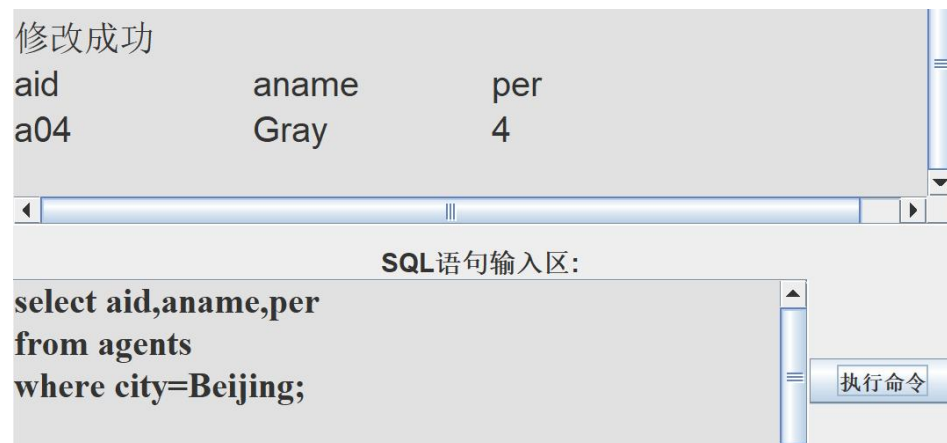
按条件查询信息



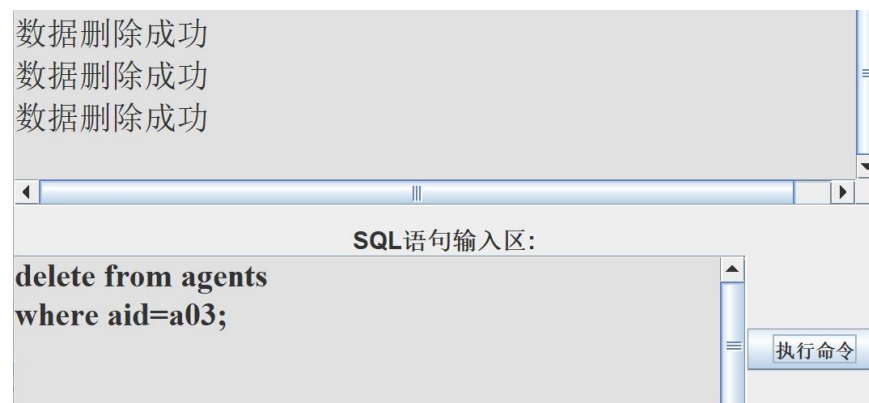
3.2.7. 更新数据



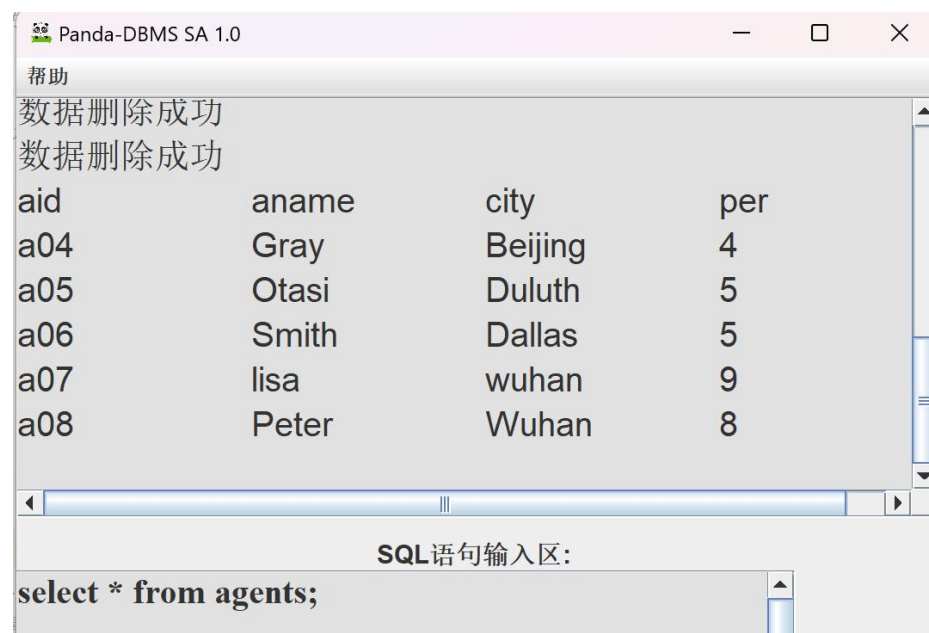
查询修改之后的数据



3. 2. 8. 删除数据

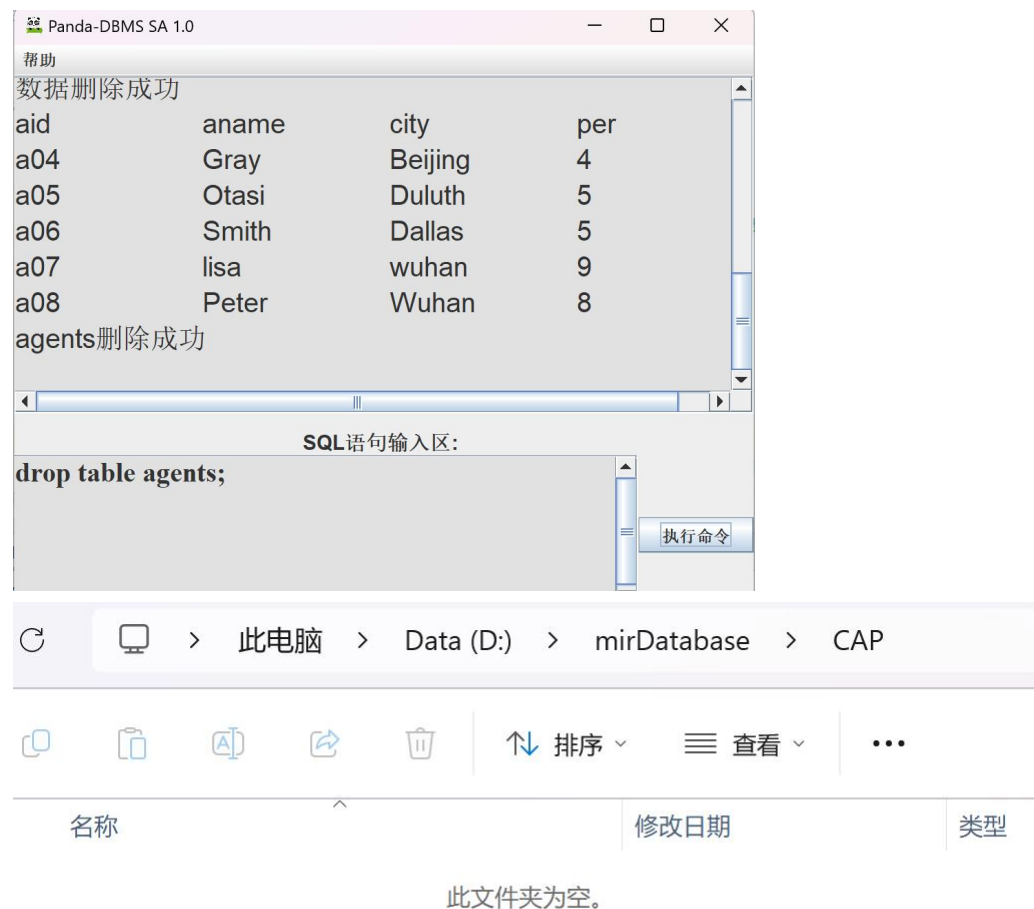


查询删除之后的数据

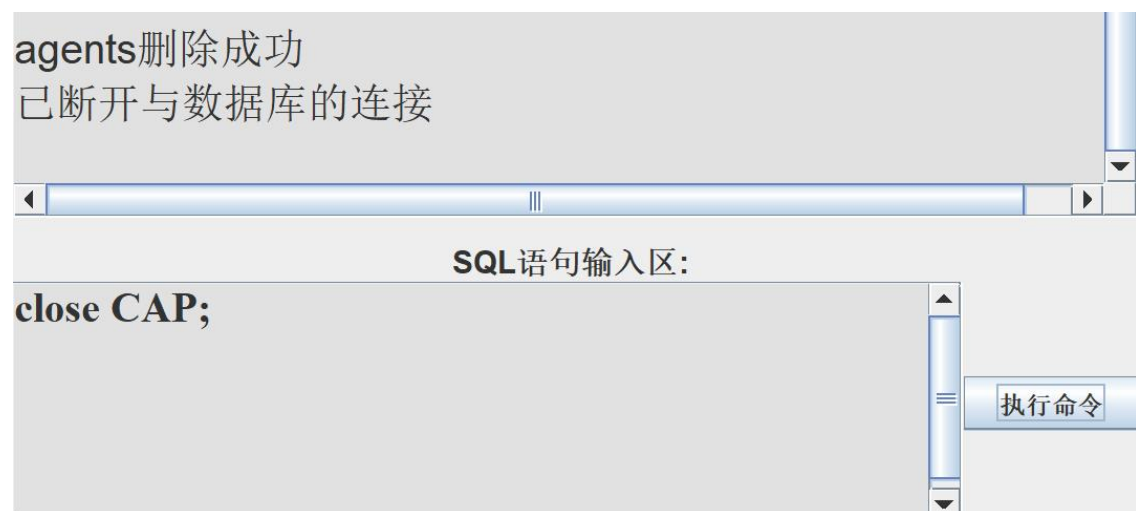


3.2.9. 删除表格

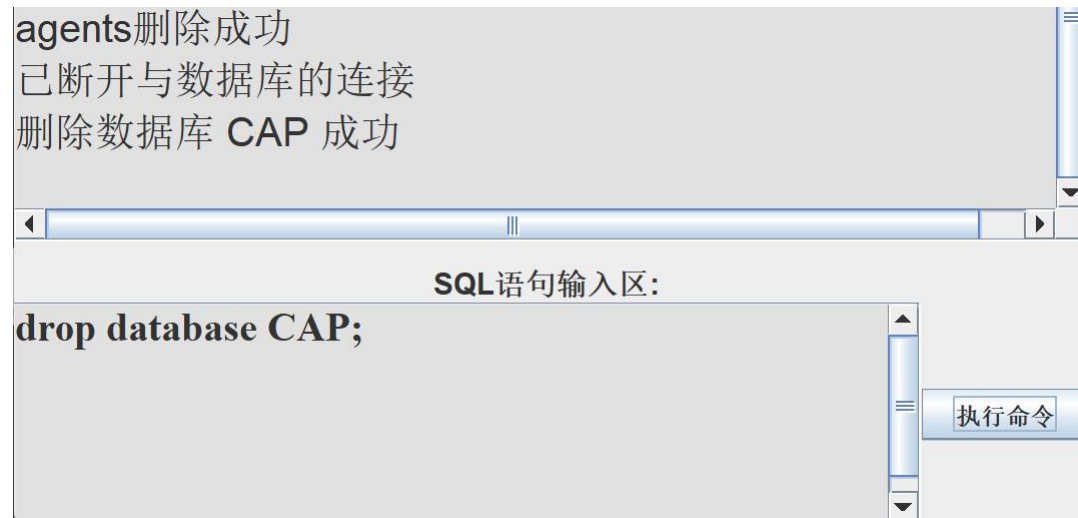
文件之中的表的信息全部删除



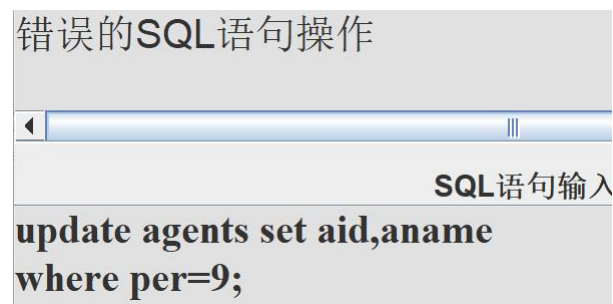
3.2.10. 关闭与数据库的连接



3.2.11. 删除数据库



3.2.12. 报错信息



四、核心代码

4.1. 登录数据库界面

验证登录密码是否登录成功

获取用户输入的用户名和密码。如果验证成功，则打开新的数据库管理窗口；如果失败，则显示错误消息。

```
@Override
public void actionPerformed(ActionEvent e) {
    Object obj = e.getSource();
    if (obj == loadBtn) {
        User user=new User();
        user.setUsername(QQmsg.getText());
        user.setPwd(new String(passwordQQ.getPassword()));
        // 验证用户名和密码是否正确
        if (usernameList.contains(user.getUsername()) && user.getPwd().equals("123456")) {
            LoadFrame.this.dispose();
            new DataBaseMangeMentFrame(user.getUsername()).setVisible(true);
        } else { // 不正确就弹出消息
            JOptionPane.showMessageDialog( parentComponent: LoadFrame.this, message: "请检查账号或密码是否正确", title: "提示", JOptionPane.WARNING_MESSAGE);
        }
    } else if(obj==messageBtn){ // 用户信息
        JOptionPane.showMessageDialog( parentComponent: LoadFrame.this, message: "用户账号:  \sa\", \"u01\", \"u02\" \n 用户密码: 123456", title: "提示", JOptionPane.INFORMA
    }
}
```

4.2. 数据库界面

执行命令

从 SQL 语句输入区 j2 中获取文本内容。根据获取的内容调用 Database.executeSQL() 方法执行相应的 SQL 操作。将执行结果显示在 j1 文本显示区中，并滚动显示区域到末尾，确保最新的信息被显示出来。

```
// 执行操作按钮事件监听
b1.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String msg = j2.getText(); // 获取文本框 j2 中的内容
        String msgpro= msg.replace( target: "\n", replacement: " ");
        System.out.println(msgpro);
        // 调用 Database.executeSQL 并获取返回信息
        Database.executeSQL(msgpro,j1);
        // 将返回的信息显示在文本显示区 j1 中
        j1.setCaretPosition(j1.getDocument().getLength());
    }
});
```


"帮助"的菜单栏

基本的菜单栏创建和事件监听器设置的示例，其中创建了一个名为 "帮助" 的菜单，里面包含了两个菜单项 "基本语法" 和 "清除显示区内容"，并为每个菜单项设置了点击事件监听器。

```
// 创建菜单栏
JMenuBar menuBar = new JMenuBar();
setJMenuBar(menuBar);

// 创建菜单
JMenu helpMenu = new JMenu(s: "帮助");
menuBar.add(helpMenu);

// 创建菜单项
JMenuItem usageItem = new JMenuItem(text: "基本语法");
helpMenu.add(usageItem);

// 添加菜单项的点击事件监听器
usageItem.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) { showUsageNotes();
});

JMenuItem clearItem = new JMenuItem(text: "清除显示区内容");
helpMenu.add(clearItem);
clearItem.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        j1.setText(""); // 清空界面
    }
});
```

4.3. 数据库 sql 语句操作语句

4.3.1. SQL 语句解析

通过循环遍历这些正则表达式，尝试匹配输入的 SQL 字符串。当匹配到某个正则表达式时，确定了对应的操作类型，并根据不同类型的操作执行相应的方法。如果无法匹配任何正则表达式，则在文本区域中显示“错误的 SQL 语句操作”。


```

public static void executeSQL(String sql, JTextArea textArea) {
    String[] bas = {
        "create database .*;",
        "drop database .*;",
        "use .*;",
        "close .*;",
        "create table .*\\(.*\\);",
        "drop table .*;",
        "select .* from .* where .*;",
        "select .* from .*;",
        "insert into .*\\(.*\\) values \\(.*\\);",
        "update +(\\w+)\\s+set\\s+(.*?)\\s+where\\s+(.*?);",
        "delete from .* where .*[>=].*;"
    };

    for (String regex : bas) {
        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(sql);
        if (matcher.find()) {
            String operation = regex.split(" ")[0];
            switch (operation) {
                case "create":
                    if (regex.contains("database")) {...} else if (regex.contains("table")) {...}
                    break;
                case "drop":
                    if (regex.contains("database")) {...} else if (regex.contains("table")) {...}
                    break;
                case "use":
                    openDatabase(sql, textArea);
                    break;
                case "close":
                    closeDatabase(sql, textArea);
                    break;
                case "select":
                    if (regex.contains("where")) {...} else {...}
                    break;
            }
        }
    }
}

```

```

        case "insert":
            insertData(sql, textArea);
            break;
        case "update":
            updateData(sql, textArea);
            break;
        case "delete":
            deleteData(sql, textArea);
            break;
        default:
            System.out.println("Operation not supported");
    }
    return;
}
}
String error = "错误的SQL语句操作\n";
textArea.append(error);
return;
}

```

4.3.2. 创建数据库

在指定路径 D:/mirDatabase/下创建一个文件夹来模拟数据库的创建。检查数据库目录是否已存在，若不存在，则创建目录并输出相应的成功，若存在则将失败信息输出到控制台的文本区域。

```

public static void createDatabase(String sql, JTextArea outputTextArea) {
    String name = getDatabaseName(sql);
    String path = "D:/mirDatabase/" + name;
    File databaseDir = new File(path);
    String out = null;
    if (!databaseDir.exists()) {
        if (databaseDir.mkdir()) {
            out = "创建成功";
            System.out.println("创建成功");
            outputTextArea.append(out + "\n");
        } else {
            out = "创建失败";
            System.out.println("创建失败");
            outputTextArea.append(out + "\n");
        }
    } else {
        out = "该数据库已经存在";
        System.out.println("该数据库已经存在");
        outputTextArea.append(out + "\n");
    }
}
}

```

4.3.3. 连接数据库

检查当前数据库连接状态

如果已经有数据库连接打开，则提示用户先关闭当前连接的数据库，并在文本区域中输出相应信息。如果数据库文件不存在，输出数据库不存在。

```
public static void openDatabase(String sql, JTextArea outputTextArea) {
    String error = null;
    if (isOpen) {
        error = "请先关闭当前连接的数据库";
        System.out.println("请先关闭当前连接的数据库");
        outputTextArea.append(error + "\n");
        return;
    }

    String name = sql.split(regex: " ")[1];
    String path = "D:/mirDatabase/";
    for (int i = 0; i < name.length() - 1; ++i){...}

    File databaseDir = new File(path);
    if (!databaseDir.exists()) {
        error = "该数据库不存在";
        System.out.println("该数据库不存在");
        outputTextArea.append(error + "\n");
    }
}
```

遍历数据库目录下的文件，将信息储存到 table 之中

创建一个 Table 对象来表示表格。对于数据库目录下的每个文件，根据文件名创建表格对象 tmpTab。读取文件内容并将其解析为表格的列信息，将其添加到表格对象中。根据数据文件是否存在，确定表格的记录数量。将解析后的表格对象添加到 tables 列表中。

```

} else {
    prePath = path;
    File[] files = databaseDir.listFiles();
    if (files != null) {
        for (File file : files) {
            if (file.getName().endsWith(".data")) {
                continue;
            }
            // 删除文件名末尾的 .txt
            String trimmedFileName = file.getName().replaceAll( regex: "\\..txt$", replacement: "");
            Table tmpTab = new Table(trimmedFileName, path: path + "/" + file.getName());
            String pathTab = path + "/" + file.getName();
            // 读取表中数据之中的信息
            try (BufferedReader br = new BufferedReader(new FileReader(pathTab))) {
                String line;
                while ((line = br.readLine()) != null) {
                    Lines tmpL = new Lines();
                    String[] columns = line.split( regex: ";");
                    int cnt = 0;
                    for (String column : columns) {
                        cnt++;
                        if (cnt == 1) {
                            tmpL.setName(column);
                        } else if (cnt == 2) {
                            tmpL.setType(column);
                        } else if (cnt == 3) {
                            if (column.matches( regex: "\\d+")) {...} else {...}
                        } else if (cnt == 4) {
                            if (column.equals("key")) {...} else {...}
                        }
                    }
                }
                tmpTab.getLines().add(tmpL); // 将 tmpL 添加到 lines 列表中
            }
        }
    }
}

```

4.3.4. 创建表格

创建新表格：

创建一个新的 Table 对象来表示新的表格。解析 SQL 命令中的表格结构信息，提取字段名称、类型、大小等信息，并构建 Lines 对象表示表格的列。将解析得到的表格列信息添加到 Table 对象中。将

```

String tableNameRegex = "create\\s+table\\s+(\\w+)\\s*\\(((.*?)\\)\\)";
Pattern pattern = Pattern.compile(tableNameRegex);
Matcher matcher = pattern.matcher(sql);
if (matcher.find()) {
    tableName = matcher.group(1);
    String fields = matcher.group(2);
    System.out.println("Table Name: " + tableName);
}

```

```

String[] fieldTokens = fields.split( regex: "," );
for (String field : fieldTokens) {
    field = field.trim();
    String[] fieldInfo = field.split( regex: "\\s+" );
    String fieldName = fieldInfo[0];
    String fieldTypeWithSize = fieldInfo[1];
    // 创建新的 Lines 对象
    Lines tmpL = new Lines();
    tmpL.setName(fieldName);
    // 移除字段类型中的括号及其内容
    String fieldType = fieldTypeWithSize.replaceAll( regex: "\\(..*?\\)", replacement: "" );
    tmpL.setType(fieldType);
    // 检查是否是关键字
    boolean isKeyField = fieldInfo.length > 2 && fieldInfo[2].equals("key");
    tmpL.setKey(isKeyField);
    // 如果字段有括号内容, 提取括号内的信息
    if (fieldTypeWithSize.contains("(") && fieldTypeWithSize.contains(")")) {
        String fieldSize = fieldTypeWithSize.substring(fieldTypeWithSize.indexOf("(") + 1, fieldTypeWithSize.indexOf(")"));
        tmpL.setSize(Integer.parseInt(fieldSize));
    }
    // 将每个字段的 Lines 对象添加到列表中
    lines.add(tmpL);
}
// 在循环外部设置最终结果到 Table 对象
tmpTab.setLines(lines);
tmpTab.setTot(tmpTab.getLines().size());
}

```

写入到.txt 文件之中

表格结构信息以特定格式写入对应的.txt 文件中。

```

String tmpPath = path + ".txt";
try (PrintWriter writer = new PrintWriter(new FileWriter(tmpPath, append: true))) {
    for (Lines line : tmpTab.getLines()) {
        StringBuilder tmpTxt = new StringBuilder();
        tmpTxt.append(line.getName()).append(";").append(line.getType()).append(";");
        if (line.getSize() != 0) tmpTxt.append(line.getSize()).append(";");
        if (line.isKey()) tmpTxt.append("key;");
        writer.println(tmpTxt.toString());
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

4.3.5. 插入数据

构建数据行

提取插入数据的列名和对应的值，并将它们分别存储在列表中。

检查提取的列名和值列表长度是否与表格结构中的列数匹配。创建一个映射（Map）来存储列名和对应的值。

// 找到现有表的名称和列名称

```
int pos = -1;
for (int i = 0; i < tables.size(); ++i) {
    if (tables.get(i).getName().equals(nameTable)) {
        pos = i;
        break;
    }
}
if (pos == -1) {...}

String columnNames = matcher.group(2);
String columnValues = matcher.group(3);
List<String> columnNameList = Arrays.asList(columnNames.split( regex: ","));
List<String> columnValueList = Arrays.asList(columnValues.split( regex: ","));
if (columnNameList.size() != tables.get(pos).getLines().size()) {
    error = "列名与表的结构不匹配";
    System.out.println("列名与表的结构不匹配");
    outputTextArea.append(error + "\n");
    return;
}
```

// 构建数据行

```
Map<String, String> columnData = new LinkedHashMap<>();
for (int i = 0; i < columnNameList.size(); i++) {
    String columnName = columnNameList.get(i).trim();
    String columnValue = columnValueList.get(i).trim();
    columnData.put(columnName, columnValue);
}
```

数据写入到.data 文件中

数据行以文本的形式到.data 文件中，实现数据的持久化存储。

// 准备写入文件之中的数据

```
StringBuilder dataLine = new StringBuilder();
for (Lines line : tables.get(pos).getLines()) {
    String value = columnData.get(line.getName());
    if (value == null) {
        System.out.println("找不到列对应的值");
        return;
    }
    dataLine.append(value).append("\t");
}

// 将数据写到文件之中
path = path + ".data";
try (PrintWriter writer = new PrintWriter(new BufferedWriter(new FileWriter(path, append: true)))) {
    writer.println(dataLine);
} catch (IOException e) {
    e.printStackTrace();
}

error = "添加数据成功";
System.out.println("添加数据成功");
outputTextArea.append(error + "\n");
```

4.3.6. 删除数据

检查数据库连接状态：

如果数据库未连接，控制台输出"未连接数据库"并直接返回，不执行后续代码。

```
public static void deleteData(String sql, JTextArea outputTextArea) {  
    String out = null;  
    if (!isOpen) {  
        out = "未连接数据库";  
        System.out.println("未连接数据库");  
        outputTextArea.append(out + "\n");  
        return;  
    }  
}
```

解析条件并过滤数据行：

从条件语句中解析出列名和值。读取.data文件的内容，根据条件过滤要删除的行，并将其余行保存在列表中。

```
// 读取 .data 文件并过滤要删除的行  
List<String> lines = new ArrayList<>();  
boolean delete = false;  
try (BufferedReader reader = new BufferedReader(new FileReader(dataFile))) {  
    String line;  
    while ((line = reader.readLine()) != null) {  
        String[] columns = line.split( regex: "\\t");  
        List<String> columnList = Arrays.asList(columns);  
        // 不添加要删除的行  
        if (columnList.contains(value)) {  
            delete = true;  
            continue;  
        }  
        lines.add(line);  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

删除数据行：

如果不存在符合条件的数据行，输出相关信息并返回。将过滤后的数据行重新写回到.data文件中。

```

if (delete == false) {
    out = "不存在该组数据";
    System.out.println("不存在该组数据");
    outputTextArea.append(out + "\n");
    return;
}
// 写入过滤后的数据回到 .data 文件
try (PrintWriter writer = new PrintWriter(new BufferedWriter(new FileWriter(dataFile)))) {
    for (String line : lines) {
        writer.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

4.3.7. 查找数据

按条件查询：

解析条件语句

检查条件字段和更新字段是否存在于表的字段中，若不存在则输出相应的错误信息并返回。

```

String dataRange = null;
String tableName = null;
String condition = null;
String pattern = "select\\s+(.*?)\\s+from\\s+(\\w+)\\s+where\\s+(.*?)";
Pattern r = Pattern.compile(pattern);
Matcher m = r.matcher(sql);
if (m.find()) {
    dataRange = m.group(1);
    tableName = m.group(2);
    condition = m.group(3);
} else {
    error = "无法解析 SQL 语句";
    outputTextArea.append(error + "\n");
    System.out.println(error);
    return;
}
File dataFile = new File( pathname: prePath + File.separator + tableName + ".data");
if (!dataFile.exists()) {...}
List<String> getL = Arrays.asList(dataRange.split( regex: ","));
String[] conditionSplit = condition.split( regex: "[<=]");
if (conditionSplit.length != 2) {
    error = "条件格式错误";
    System.out.println("条件格式错误");
    outputTextArea.append(error + "\n");
    return;
}

```


根据处理数据范围和条件选择要显示的数据，读取数据文件并且选择数据并将其放入 showData 列表中。

```
// 读取文件选择要操作的数据
List<String> showData = new ArrayList<>();
try (BufferedReader reader = new BufferedReader(new FileReader(dataFile))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] columns = line.split("regex: \"\\t\"");
        StringBuilder tmp = new StringBuilder();
        boolean isShow = false;
        List<String> con = new ArrayList<>();
        for (int j = 0; j < tables.get(pos).getLines().size(); ++j) {
            String tmpStr = columns[j];
            boolean b = tables.get(pos).getLines().get(j).getName().equals(gl);
            if(b &&dayu){
                Integer infin=Integer.parseInt(inf);
                Integer tmpin=Integer.parseInt(tmpStr);
                if(tmpin>infin){
                    isShow=true;
                }
            }
            if(b &&xiaoyu){
                Integer infin=Integer.parseInt(inf);
                Integer tmpin=Integer.parseInt(tmpStr);
                if(tmpin<infin){
                    isShow=true;
                }
            }
            if (dengyu) {
                if (b && tmpStr.equals(inf)) {
                    isShow = true;
                }
            }
            for (int k : checkLine) {
                if (j == k) {
                    con.add(tmpStr);
                    break;
                }
            }
        }
        if (isShow) {
            tmp.append(String.join("delimiter: \"\\t\", con));
            showData.add(tmp.toString());
        }
    }
} catch (IOException e) {
    e.printStackTrace();
}

StringBuilder columnsOutput = new StringBuilder();
for (String column : getL) {
    columnsOutput.append(column).append("\\t");
}
columnsOutput.append("\\n"); // 添加换行
outputTextArea.append(columnsOutput.toString()); // 将列名输出到文本区域

// 输出数据到文本区域
for (String data : showData) {
    outputTextArea.append(data + "\\n"); // 输出数据到文本区域，并在每条数据后添加换行
}
```

查询整个表中的数据

```
// 找到对应的表
Table selectedTable = tables.get(pos);
List<String> tmpData = new ArrayList<>();
try (BufferedReader reader = new BufferedReader(new FileReader(fileName: prePath + "/" + selectedTable.getName() + ".data"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        tmpData.add(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}

// 输出列名到文本区域
StringBuilder columnsOutput = new StringBuilder();
for (Lines line : selectedTable.getLines()) {
    columnsOutput.append(line.getName()).append("\t");
}
columnsOutput.append("\n");
outputTextArea.append(columnsOutput.toString());

for (String data : tmpData) {
    outputTextArea.append(data + "\n");
}
outputTextArea.repaint(); // 刷新文本区域显示
```

4.3.8. 更新数据

解析 SQL 语句:

使用正则表达式提取表名和更新的字段和更新值。

```
String pattern = "update\\s+(\\w+)\\s+set\\s+(.*?)\\s+where\\s+(.*?)";
Pattern r = Pattern.compile(pattern);
Matcher m = r.matcher(sql);
String nameTab = null;
String tmpInf = null;
if (m.find()) {
    nameTab = m.group(1); // 提取表名
    tmpInf = m.group(3); // 提取条件
} else {
    System.out.println("无效的 SQL 语句");
    return;
}

String[] updates = m.group(2).split(regex: ","); // 按逗号分割多个字段的更新
Map<String, String> updateMap = new LinkedHashMap<>();
for (String fieldUpdate : updates) {
    String[] fieldValuePair = fieldUpdate.trim().split(regex: "=");
    if (fieldValuePair.length == 2) {
        String fieldName = fieldValuePair[0].trim();
        String fieldValue = fieldValuePair[1].trim().replaceAll(regex: "'", replacement: "");
        updateMap.put(fieldName, fieldValue);
    }
}

String[] conditionSplit = tmpInf.split(regex: "<|>|<=|>=");
if (conditionSplit.length != 2) {
    System.out.println("条件格式错误");
    return;
}

String nameL = conditionSplit[0].trim();
String inf = conditionSplit[1].trim().replaceAll(regex: "'", replacement: "");
```

执行更新操作：

读取文件中的数据行，并根据条件更新数据。若条件字段的值符合条件，则更新对应的字段值，更新后的数据写入到临时数据列表中。

```
List<String> tmpData = new ArrayList<>();
try (BufferedReader reader = new BufferedReader(new FileReader(dataFile))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] columns = line.split(regex: "\\t");
        Map<String, String> newColumn = new LinkedHashMap<>();
        List<String> columnList = Arrays.asList(columns);

        for (int i = 0; i < columnList.size(); i++) {
            String cName = tables.get(pt).getLines().get(i).getName().trim();
            String columnValue = columnList.get(i).trim();
            newColumn.put(cName, columnValue);
        }

        if (columnList.get(pl).equals(inf)) {
            for (String key : updateMap.keySet()) {
                newColumn.put(key, updateMap.get(key));
            }
        }

        StringBuilder dataLine = new StringBuilder();
        for (String cName : newColumn.keySet()) {
            String value = newColumn.get(cName);
            dataLine.append(value).append("\\t");
        }
        tmpData.add(dataLine.toString());
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

4.4. 工具类

4.4.1. Table 类

Table 类表示数据库中表的结构，它具有存储表的基本属性和列信息

```

public class Table {
    3 个用法
    private String name; // 表的名称
    3 个用法
    private String path; // 表的存储路径
    2 个用法
    private int tot; // 表内数据的总数
    3 个用法
    private List<Lines> lines; // 表的列信息

    // 构造函数
    1 个用法
    public Table(String name, String path) {
        this.name = name;
        this.path = path;
        this.lines = new ArrayList<>();
    }
    // 空参构造
    1 个用法
    public Table() {
    }
}

```

4.4.2. Lines 类

该类表示数据库表中的列信息，可以用于存储和管理列的属性。

```

public class Lines {
    3 个用法
    private String name; // 列的名称
    3 个用法
    private String type; // 列的数据类型
    3 个用法
    private int size; // 列的大小
    3 个用法
    private boolean isKey; // 是否为主键

    // 构造函数
    2 个用法
    public Lines(){
    }
    0 个用法
    public Lines(String name, String type, int size, boolean isKey) {
        this.name = name;
        this.type = type;
        this.size = size;
        this.isKey = isKey;
    }
}

```

4.5. 解决难点

SQL 语句解析

使用正则表达式两次检查 SQL 指令：

第一次检查语法错误。

第二次进行分词，检查指令是否可执行。

分成两个独立的函数处理，降低臃肿性，提高容错率。

文件操作：

JAVA 对文件操作复杂，导致部分操作无法简洁进行。

基于此读入整个文件内容，修改后再输出。

文件数据的查询采用的列表和 hash 表等方法来存储

该 DBMS 通过文件存储实现数据的离线存储与处理，在 SQL 语句解析和文件操作方面克服了一些挑战，虽然存在一些限制，但成功实现了基本的数据库操作功能。